

PCAD

Programmazione Concorrente

Algoritmi Distribuiti

Arnaud Sangnier
arnaud.sangnier@unige.it

Mutua esclusione ancora e per più processi

Storia

- Algoritmo di Dekker: 1964
- Algoritmo di Peterson: 1981
 - Più facile di dimostrare la sua correttezza
 - Più facile da adattare per $N > 2$ processi

Problema della sezione critica

Algoritmo di Peterson

```
int turn=1; //variabili condivise  
bool D1=false;  
bool D2=false;
```

```
Process P1  
while(true){  
p1: SNC  
p2: D1=true;  
p3: turn=2;  
p4: while(turn!=1 &&D2==true){}  
p5: SC  
p6:D1=false;}
```

```
Process P2  
while(true){  
q1: SNC  
q2: D2=true;  
q3: turn=1;  
q4: while(turn!=2 &&D1==true){}  
q5: SC  
q6:D2=false;}
```

Problema della sezione critica

Algoritmo di Peterson - Proprietà

Teorema:

L'algoritmo di Peterson verifica la mutua esclusione.

Prova:

- Supponiamo che P1 arrivi in SC dopo P2
- Allora D2=D1=true (P1 e P2 sono in p4 e p5)
 - e turn=1 (perché P1 è uscito dal while)
- Come ha fatto P2 a passare il while ?
 - **con D1=false ?**
 - dunque nel frattempo P1 ha impostato D1=true e turn=2
 - ma allora P1 non può passare il while
 - **con turn=2 ?**
 - Allora turn rimane sempre uguale a 2 finché P2 è in SC e P1 non può passare il while

```
Process P1
while(true){
p1: SNC
p2: D1=true;
p3: turn=2;
p4: while(turn!=1 &&D2==true){}
p5: SC
p6:D1=false;}
```

Problema della sezione critica

Algoritmo di Peterson - Proprietà

Teorema:

L'algoritmo di Peterson verifica l'assenza di deadlock.

Prova:

- Se c'è deadlock, P1 è bloccato in p4 e P2 in q4
- Abbiamo
- $(\text{turn} \neq 1 \ \&\& \ D2 == \text{true}) \ \&\& \ (\text{turn} \neq 2 \ \&\& \ D1 == \text{true})$
 - IMPOSSIBILE perché turn prende come valore 1 o 2!!!

```
Process P1
while(true){
p1: SNC
p2: D1=true;
p3: turn=2;
p4: while(turn!=1 &&D2==true){}
p5: SC
p6:D1=false;}
```

Problema della sezione critica

Algoritmo di Peterson - Proprietà

Teorema:

L'algoritmo di Peterson verifica l'assenza di starvation sotto le seguenti ipotesi:
-equità fra i processi (se un processo può eseguire un'istruzione, la eseguirà prima o poi)
-la SC termina sempre e finisce con l'esecuzione del post-protocollo

Prova:

- Supponiamo starvation di P1, allora
 - D1=true per sempre
 - e D2=true e turn=2 infinitamente spesso
(ogni volta che lo scheduler darà la mano a P1)
 - D2=true => P2 è in q3,q4,q5,q6
 - Se è in q3, mette turn a 1 => P1 proseguirà ✗
 - Se è in q4 e turn=1,=> P1 proseguirà ✗
 - Se è in q4 e turn=2, andrebbe in q5 e ...
 - Se è in q5, andrebbe in q6, e ...
 - Se è in q6, andrebbe in q1, poi in q2 e in q3!

```
Process P1
while(true){
  p1: SNC
  p2: D1=true;
  p3: turn=2;
  p4: while(turn!=1 &&D2==true){}
  p5: SC
  p6:D1=false;}
```

Passare da 2 a $N > 2$ processi

- Diversi algoritmi per passare da 2 a N processi, vedremmo:
 - the filter lock
 - ci sono $N-1$ sale d'attesa che bloccano al meno un processo
 - bisogna ad attraversare queste sale
 - il torneo
 - ad ogni turno, il numero massimo di processi che va avanti è diviso per 2

Idea dietro the Filter Lock

- Algoritmo per $N > 2$ processi
- Ci sono $N-1$ sale d'attesa da attraversare
- Per ogni sala (**level**):
 - Tra i processi che provano a passare, **almeno uno riesce**
 - Se ci sono più processi che provano, **almeno uno viene bloccato**

Problema della sezione critica

Algoritmo di Peterson

```
int turn=1; //variabile condivisa  
bool D1=false;  
bool D2=false;
```

```
Process P1  
while(true){  
p1: SNC  
p2: D1=true;  
p3: turn=2;  
p4: while(turn!=1 &&D2==true){}  
p5: SC  
p6:D1=false;}
```

```
Process P2  
while(true){  
q1: SNC  
q2: D2=true;  
q3: turn=1;  
q4: while(turn!=2 &&D1==true){}  
q5: SC  
q6:D2=false;}
```

Chi ha turn è prioritario!!!

Problema della sezione critica

Algoritmo di Peterson - Cambiamento punto di vista

```
int victim=1; //variabile condivisa  
bool D1=false;  
bool D2=false;
```

```
Process P1  
while(true){  
p1: SNC  
p2: D1=true;  
p3: victim=1;  
p4: while(victim==1 &&D2==true){}  
p5: SC  
p6:D1=false;}
```

```
Process P2  
while(true){  
q1: SNC  
q2: D2=true;  
q3: victim=2;  
q4: while(victim==2 &&D1==true){}  
q5: SC  
q6:D2=false;}
```

Self mutilation, la vittima viene bloccata!!!

Problema della sezione critica

The Filter Lock

Processi $P[0] \dots P[N-1]$

$\text{int level}[N] = \{0, \dots, 0\};$ //variabili condivise

$\text{int victim}[N] = ?$

```
Process P[id]
while(true){
p1: SNC
p2: for (int j=1; j<N; j++){
p3:  level[id]=j
p4:  victim[j]=id
p5:  while(  $\exists k \neq \text{id}. \text{level}[k] \geq j$  && victim[j]==id){}
      }
p6: SC
p7: level[id]=0;}
```



**Un processo può andare avanti se non
è la vittima o se non c'è nessuno ad un
livello superiore**

Problema della sezione critica

Filter Lock - Proprietà

Definizione:

- Un processo ha passato il livello k se ha passato p5 per $j=k$ (o se $k=0$)
- $\text{num}_{\geq k}$: numero di processi che ha passato il livello k

```
Process P[id]
while(true){
p1: SNC
p2: for (int j=1; j<N; j++){
p3:  level[id]=j
p4:  victim[j]=id
p5:  while(  $\exists k \neq \text{id} . \text{level}[k] \geq j \ \&\& \ \text{victim}[j] == \text{id}$  ){
      }
p6: SC
p7: level[id]=0;}
```

Problema della sezione critica

Filter Lock - Proprietà

Proprietà:

Abbiamo $\text{num}_{\geq j} \leq n-j$ for all $0 \leq j \leq n-1$.

Prova (per induzione) :

- **j=0:** si abbiamo $\text{num}_{\geq 0} \leq n$
- **j>0:**
 - per ipotesi di induzione, sappiamo che al massimo $n-j+1$ processi hanno passato il livello $j-1$
 - Supponiamo che tutti hanno passato il livello j
 - Sia M , l'ultimo ad avere fatto $\text{victim}[j]=M$
 - Tutti gli altri M' , hanno fatto prima
 - $\text{level}[M']=j$
 - $\text{victim}[j]=M'$
 - **Non è possibile che M abbia passato il while!!**

```
Process P[id]
while(true){
p1: SNC
p2: for (int j=1; j<N; j++){
p3:  level[id]=j
p4:  victim[j]=id
p5:  while(  $\exists k \neq \text{id} . \text{level}[k] \geq j \ \&\& \ \text{victim}[j] == \text{id}$  ){
      }
p6: SC
p7: level[id]=0;}
```

Problema della sezione critica

Filter Lock - Proprietà

Proprietà:

Abbiamo $\text{num}_{\geq j} \leq n-j$ for all $0 \leq j \leq n-1$.

Corollario:

$\text{num}_{\geq (n-1)} \leq 1$

Teorema:

L'algoritmo di Filter Lock verifica la mutua esclusione.

Problema della sezione critica

Filter Lock - Proprietà

Teorema:

L'algoritmo di Filter Lock verifica l'assenza di starvation sotto gli ipotesi:

- equità fra i processi (se un processo può eseguire una istruzione, la eseguirà un giorno)
- la SC termina sempre e finisce con l'esecuzione del post-protocollo

Proprietà:

Ogni processo al livello $j \geq 1$ arriverà al livello $j+1$ o in SC se $j=N-1$.

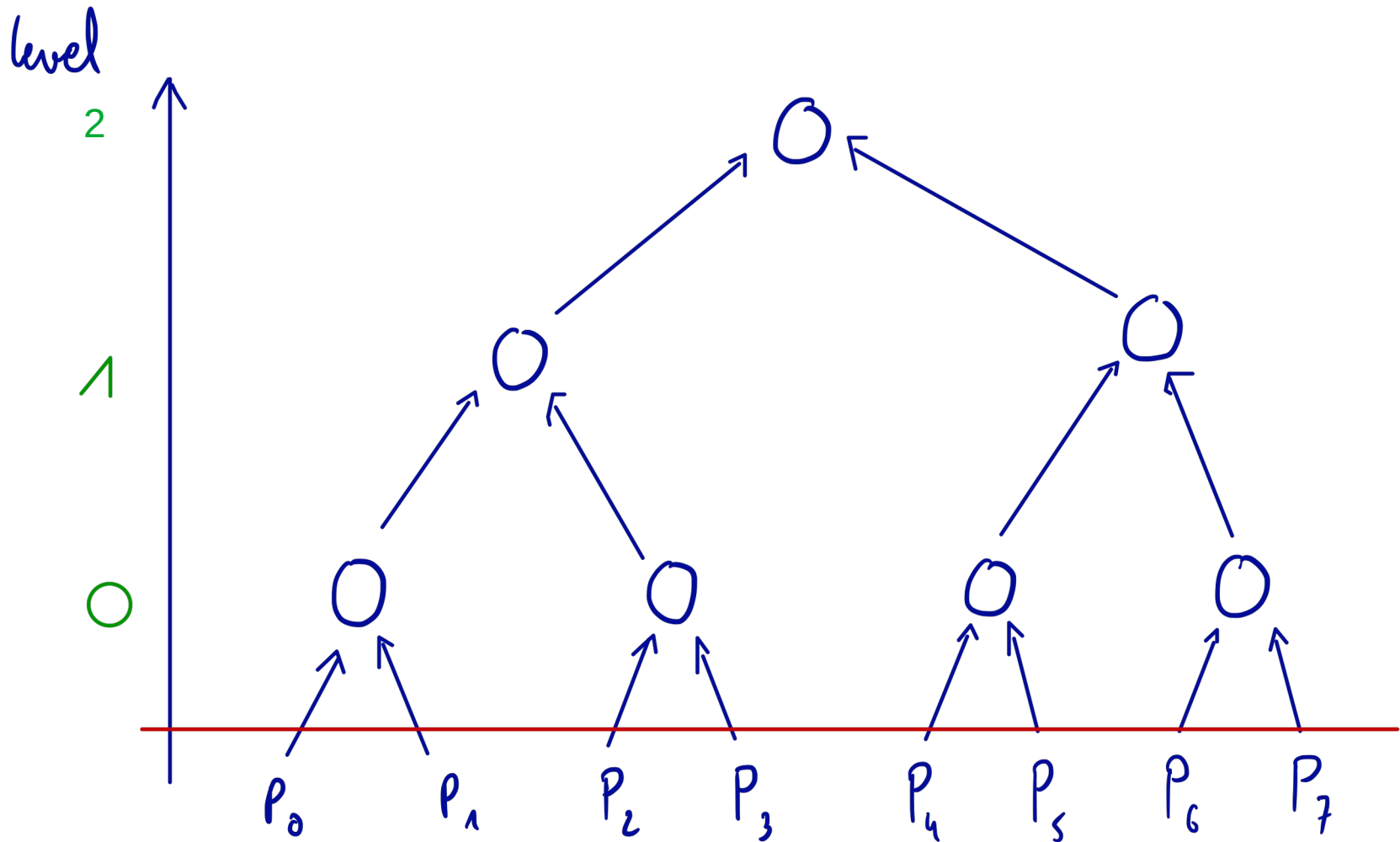
Prova (per induzione rovesciata):

- $j=N-1 \rightarrow$ OK, come Peterson, abbiamo al massimo due processi
- $j < N-1$: Sia M al livello j bloccato nel while
 - Per ipotesi, gli processi dei livelli superiori arrivano in CS
 - Nessun processo del livello $j-1$ può arrivare al livello j (altrimenti victim cambia e M è sbloccato). Quindi `victim[j]` non cambia.
 - M è dunque l'unico ad essere bloccato al livello j e dopo il 'successo' degli altri verrà per forza sbloccato.

Il torneo

- Abbiamo $n=2^k$ processi : $P_0, P_1, \dots, P_n$
- Facciamo un torneo con eliminazione in $\log(n)=k$ giri
- Dopo il primo giro, sono 'eliminati' $n/2$ processi
 - Ogni competizione fra due processi è gestita da un algoritmo di mutua esclusione per due processi
- Dopo il secondo giro, sono 'eliminati' $n/4$ processi
- etc
- I giri sono numerati da 0 a $\log(n)-1$

Il torneo



Il torneo

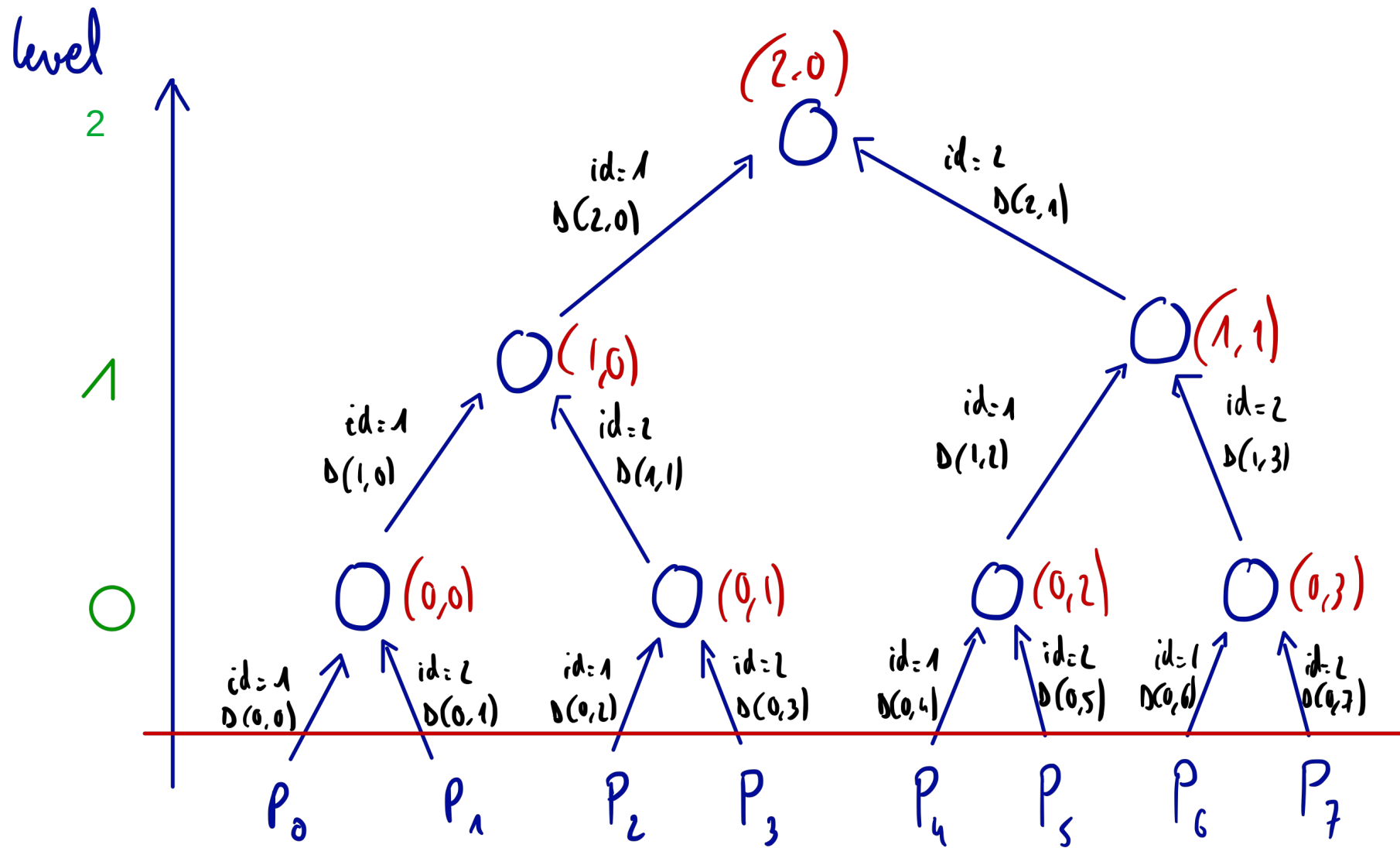
- Ogni 'partita' è identificata da un giro/livello l e da un numero k con :
 - $le \in \{0, \dots, \log(n)-1\}$ e $k \in \{0, \dots, 2^{\log(n)-1-le}-1\}$
- Per usare l'algoritmo di Peterson, dobbiamo avere l'equivalente delle variabili condivise:
 - $\text{int turn}(le, k)$: chi è prioritaria per la partita
 - $\text{bool D}(le, 2k)$ il processo di **sinistra** vuole accedere alla sezione critica
 - $\text{bool D}(le, 2k+1)$ il processo di **destra** vuole accedere alla sezione critica
- Ogni processo deve anche sapere se è il processo di sinistra o di destra nella partita che sta giocando
- Per ogni processo, abbiamo quindi :
 - le : il giro corrente
 - k : il numero della partita nel giro l
 - $\text{id} \in \{1, 2\}$ che indica se il processo è a sinistra (1) o a destra (2)

Problema della sezione critica

il torneo

```
Process P[i]
while(true){
p1: SNC
p2: k=i
p3: for(int le=0; le<log(n); le++){
p4:   id=k%2+1 //id vale 1 o 2
p5:   k=k/2
p6:   D(le,2k+(id-1))=true
p7:   turn(le,k)=3-id
p8:   while(turn(le,k)!=id &&D(le,2k+2-id)==true){}
      }
p9: SC
p10: for(int le=log(n)-1; le>=0; le--){
p11:   D(le,i/2le)=false}
```

Il torneo



Problema della sezione critica

Algoritmo del torneo - Proprietà

Teorema:

L'algoritmo del torneo verifica la mutua esclusione.

Prova:

- Se due processi sono insieme in SC, sono passati entrambi dalla stessa partita in un determinato momento
- Sia (l, k) quella partita.
- Poiché i due processi provengono da due rami diversi hanno due id diversi
- Tuttavia se uno ha superato il while e l'altro l'ha raggiunto senza eseguire il post-protocollo, significa che l'algoritmo di Peterson è falso....

Problema della sezione critica

Algoritmo del torneo - Proprietà

Teorema:

L'algoritmo del torneo Lock verifica l'assenza di starvation sotto gli ipotesi:

- equità fra i processi (se un processo può eseguire una istruzione, la eseguirà un giorno)
- la SC termina sempre e finisce con l'esecuzione del post-protocollo

Prova:

- Se un processo è bloccato alla partita (l, k) , a un certo punto diventerà prioritario, una volta che l'altro processo partecipante alla partita avrà eseguito il suo post-protocollo

Storia

- Algoritmo del fornaio (*bakery algorithm*): algoritmo di mutua esclusione per $N \geq 2$ processi
- Inventato da Leslie Lamport in 1974
- Usa delle variabili **proprie** (multiple readers/single writer)
 - ogni processo ha le sue variabili che possono essere lette ma non modificate dagli altri processi
- È l'algoritmo usato nell'amministrazione, o dalle banche:
 - un nuovo 'cliente' che vuole avere accesso alle risorse prende un numero superiore ai numeri dei clienti già in attesa
 - il cliente prioritario è quello che ha il numero più piccolo

Problema della sezione critica

Algoritmo del fornaio

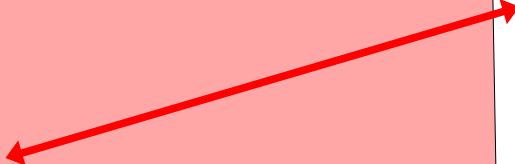
Processi $P[0] \dots P[N-1]$

bool choosing[N]={false,...,false}; //variabile condivise

int nb[N]={0,...,0};

```
Process P[id]
while(true){
p1: SNC
p2: choosing[id]=true;
p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
p4: choosing[id]=false;
p5: for (int j=0; j<N; j++){
p6:   if(j!=id){
p7:     while(choosing[j]){
p8:       while(nb[j]!=0 && (nb[j],j) << (nb[id],id)){}
p9:   }
p10: nb[id]=0
```

**NB: il calcolo del
Max non è atomico**



Nuova relazione totale di ordinamento:

$(nb[j],j) << (nb[id],id)$ sse $((nb[j]<nb[id])$ o $(nb[j]==nb[id]$ e $(j<id))$

Problema della sezione critica

Algoritmo del fornaio - Proprietà

Teorema:

L'algoritmo del fornaio verifica la mutua esclusione.

Prova:

- Scegliamo un processo $P[i]$ che arriva in SC ad un istante t_2
- Dobbiamo dimostrare che a questo istante, nessun altro processo $P[k]$ (con $k \neq i$) è in SC
- Scegliamo un $k \neq i$ e vediamo dove è $P[k]$ in t_2
- Usiamo due altri istanti:
 - t_0 : $P[i]$ passa il
while(choosing[k]){}
 - t_1 : $P[i]$ passa il
while(nb[k] != 0 && (nb[k], k) << (nb[i], i)){}
 - t_2 : $P[i]$ arriva in SC

```
Process P[id]
while(true){
  p1: SNC
  p2: choosing[id]=true;
  p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
  p4: choosing[id]=false;
  p5: for (int j=0; j<N; j++){
  p6:   if(j!=id){
  p7:     while(choosing[j]){}
  p8:     while(nb[j]!=0 && (nb[j],j) << (nb[id],id)){}
  p9:   }
  p10: nb[id]=0
```

Problema della sezione critica

Algoritmo del fornaio - Proprietà

- Dove può essere $P[k]$ a t_0 ?
 - **in p3,p4 ?** no perché a t_0 , $P[i]$ passa `while(choosing[k]){} quindi`
`choosing[k]=false`
 - **in p1,p2 ?** quindi quando $P[k]$ eseguirà il suo pre-protocollo, non potrà essere in SC prima che $P[i]$ ne esca

```
Process P[id]
while(true){
p1: SNC
p2: choosing[id]=true;
p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
p4: choosing[id]=false;
p5: for (int j=0; j<N; j++){
p6:   if(j!=id){
p7:     while(choosing[j]){}
p8:     while(nb[j]!=0 && (nb[j],j) << (nb[id],id)){}
p9:   }
p10: nb[id]=0
```

Problema della sezione critica

Algoritmo del fornaio - Proprietà

- Dove può essere $P[k]$ a t_0 ?
 - **in $p_5, p_6, p_7, p_8, p_9, p_{10}$?** ma allora $nb[k]$ è fisso e abbiamo due casi

1) $(nb[k], k) << (nb[i], i)$

- allora $nb[k]$ deve essere messo a 0 prima di t_1
- $P[k]$ passerà quindi in p_1, p_2 (come prima)

2) $(nb[i], i) << (nb[k], k)$

- il calcolo di $nb[i]$ inizia prima della fine di quello di $nb[k]$
- con $\text{while}(\text{choosing}[i])\{ \}$ $P[k]$ dovrà aspettare che $nb[i]$ sia calcolato e verrà bloccato in p_8

$P[k]$ non può essere in SC in t_2

```
Process P[id]
while(true){
p1: SNC
p2: choosing[id]=true;
p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
p4: choosing[id]=false;
p5: for (int j=0; j<N; j++){
p6:   if(j!=id){
p7:     while(choosing[j]){
p8:       while(nb[j]!=0 && (nb[j],j) << (nb[id],id)){}
p9:   }
p10: nb[id]=0
```

Problema della sezione critica

Algoritmo del fornaio - Proprietà

Teorema:

L'algoritmo del fornaio verifica l'assenza di deadlock

Prova:

- Se tanti processi si bloccano, si ritrovano tutti bloccati in p8
- Ma dato che l'ordine $<<$ è totale (ogni processo ha un numero diverso), c'è per forza un i tale quale $(nb[i], i)$ è l'elemento più piccolo e lui passerà
=> quindi **non ci sarà un deadlock**

```
Process P[id]
while(true){
p1: SNC
p2: choosing[id]=true;
p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
p4: choosing[id]=false;
p5: for (int j=0; j<N; j++){
p6:   if(j!=id){
p7:     while(choosing[j]){
p8:       while(nb[j]!=0 && (nb[j],j) << (nb[id],id)){}
p9:     SC
p10: nb[id]=0
```

Problema della sezione critica

Algoritmo del fornaio - Proprietà

Teorema:

L'algoritmo del fornaio verifica l'assenza di starvation sotto gli ipotesi:

- equità fra i processi (se un processo può eseguire una istruzione, la eseguirà un giorno)
- la SC termina sempre e finisce con l'esecuzione del post-protocollo

Prova:

- Supponiamo che $P[i]$ rimanga bloccato nel suo pre-protocollo
- È bloccato per forza in p8 per un j , i.e. $nb[j] > 0 \ \&\& \ (nb[j], j) << (nb[i], i)$
- $P[j]$ finirà in SC (non c'è deadlock) e dopo:
 - $P[j]$ rimarrà in SC, e $nb[j] = 0$
 - $P[j]$ rifarà il suo pre-protocollo e il calcolo di $nb[j]$ prenderà in considerazione il valore di $nb[i]$ e $P[i]$ passerà

```
Process P[id]
while(true){
p1: SNC
p2: choosing[id]=true;
p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
p4: choosing[id]=false;
p5: for (int j=0; j<N; j++){
p6:   if(j!=id){
p7:     while(choosing[j]){
p8:       while(nb[j]!=0 && (nb[j],j) << (nb[id],id)){}
p9:   }
p10: nb[id]=0
```

Problema della sezione critica

Algoritmo del fornaio - Proprietà

Teorema:

L'algoritmo del fornaio verifica l'assenza di starvation sotto gli ipotesi:

- equità fra i processi (se un processo può eseguire una istruzione, la eseguirà un giorno)
- la SC termina sempre e finisce con l'esecuzione del post-protocollo

Prova:

- Assumiamo che $P[i]$ rimane bloccato nel suo pre-protocollo
- È per forza bloccato in p8 per un j , i.e. $nb[j]>0 \ \&\& \ (nb[j],j) << (nb[i],i)$
- **$P[j]$ finirà in SC (non c'è deadlock)** e dopo:
 - $P[j]$ rimarrà in SC, e $nb[j]=0$
 - $P[j]$ rifarà il suo pre-protocollo e il
 - calcolo di $nb[j]$ prenderà in
 - considerazione il
 - valore di $nb[i]$ e $P[i]$ passerà



In realtà, bisogna ad iterare questo passo

```
Process P[id]
while(true){
  p1: SNC
  p2: choosing[id]=true;
  p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
  p4: choosing[id]=false;
  p5: for (int j=0; j<N; j++){
  p6:   if(j!=id){
  p7:     while(choosing[j]){
  p8:       while(nb[j]!=0 && (nb[j],j) << (nb[id],id)){}
  p9:     }
  p10: nb[id]=0
  }
```

Problema della sezione critica

Algoritmo del fornaio - Proprietà

Teorema:

Nel l'algoritmo del fornaio i valori nel nb possono crescere all'infinito (anche con solo due processi)!

```
Process P[id]
while(true){
p1: SNC
p2: choosing[id]=true;
p3: nb[id]=Max(nb[0],...,nb[n-1])+1;
p4: choosing[id]=false;
p5: for (int j=0; j<N; j++){
p6:   if(j!=id){
p7:     while(choosing[j]){}
p8:     while(nb[j]!=0 && (nb[j],j) << (nb[id],id)){}
p9:   }
p10: nb[id]=0
```

Altri metodi basati su 'tecnologie' più avanzate che variabili condivise

Test-and-set

- Un test-and-set è una variabile booleana con due operazioni atomiche:
 - 1) **test-and-set**: legge il valore della variabile, la mette a true, e ritorna il valore letto
 - 2) **reset**: mette false nella variabile

Algoritmo per mutua esclusione

test-and-set x=false; //variabile condivisa

```
Process P
while(true){
  p1: SNC
  p2: while(x.test-and-set()==true){}
  p5: SC
  p6: x.reset()}
```

Test-and-set

- Un test-and-set è una variabile boolean con due operazione atomiche:
 - 1) **test-and-set**: legge il valore della variabile, la mette a true, e restituisce il valore letto
 - 2) **reset**: imposta il valore della variabile su false

Algoritmo per mutua esclusione

test-and-set x=false; //variabile condivisa

```
Process P
while(true){
p1: SNC
p2: while(x.test-and-set()==true){}
p3: SC
p4: x.reset()}
```

- Mutua esclusione 
- Assenza di deadlock 
- Assenza di starvation 
- Attesa limitata 